

# Project Moonslide

## Mid-Evaluation Report

### Mentors:

Mihir, Shreyansh, Tanmay, Tripti

### Mentees:

Anuj, Arnav, Harshit, Kartik,  
Mayank, Niharika, Sayantan, Sheetal

### Abstract

**Project Moonslide** aims to detect and characterise landslides on the lunar surface using high-resolution Chandrayaan-II imagery from the OHRC (Optical High Resolution Camera) and TMC-2 (Terrain Mapping Camera 2) instruments. The project integrates satellite remote sensing, image pre-processing, GIS-based spatial analysis, and machine-learning methods—particularly Convolutional Neural Networks (CNNs)—to identify landslide-prone crater regions, extract geometric and spatial insights, and probe recent lunar geological activity. This mid-evaluation report documents the background astronomy and planetary science theory taught during the first half of the project, the machine-learning and computer-vision foundations introduced to the mentees, and the results of a hands-on CNN image-classification task completed by the group. Chandrayaan-II datasets have been accessed and explored via the PRADAN portal; the next phase will apply CNN-based detection directly to OHRC and TMC-2 lunar imagery in conjunction with QGIS-based geo-spatial analysis.

## 1. ABOUT THE PROJECT

**Project Moonslide** is an 8 week long mentorship project conducted by the Astronomy Club, IIT Kanpur. The project bridges observational planetary science with modern computational methods to study mass-wasting events on the Moon.

### Core scientific objectives:

- Detect landslides on the Moon using high-resolution Chandrayaan imagery.
- Use landslides as proxies to study recent lunar geological activity.
- Develop image-processing and ML-based algorithms for feature detection across varied terrains.
- Extract insights such as geometry, spatial distribution, and source regions.
- Combine satellite data with computational methods to map geologically active regions.
- Gain an understanding of lunar surface evolution and dynamics.

### Technical skills developed:

- Understanding lunar surface geology and landslide formation mechanisms.

- Working with satellite imagery from Chandrayaan datasets (TMC, DTM, OHRC).
- Image pre-processing and enhancement techniques.
- Basics of computer vision: edge detection, segmentation, pattern recognition.
- Introduction to GIS tools: QGIS and ArcMap.
- Machine Learning concepts for image processing, focusing on CNNs and their history.

## 2. CELESTIAL SPHERE & COORDINATE SYSTEMS

The **celestial sphere** is an imaginary sphere of infinite radius, concentric with the Earth, onto which all celestial objects are projected. It is a purely geometric construct that serves as an indispensable visualisation tool for locating objects in the sky and describing the apparent motions of celestial bodies.

To uniquely specify the position of any object on the celestial sphere, astronomers define a set of **coordinate systems**, each tied to a different reference plane and reference direction. The appropriate system is chosen based on the observation context.

### 2.1 1. Horizontal Coordinate System (Alt-Az)

This system is observer-centric and time-dependent. Its reference plane is the *local horizon* of the observer. Because it depends on both the location and time of the observer, the same star will have different coordinates for two observers at different locations or different times. It is also called the **Alt-Az Coordinate System**.

- **Altitude** — the angular distance of the object measured *above* or *below* the horizon along a vertical circle.  $+90^\circ$  = Zenith (directly overhead);  $0^\circ$  = on the horizon;  $-90^\circ$  = Nadir (directly below).
- **Azimuth** — the angular distance measured *clockwise* from North along the horizon to the foot of the vertical circle passing through the object.  $0^\circ$  = North;  $90^\circ$  = East;  $180^\circ$  = South;  $270^\circ$  = West.

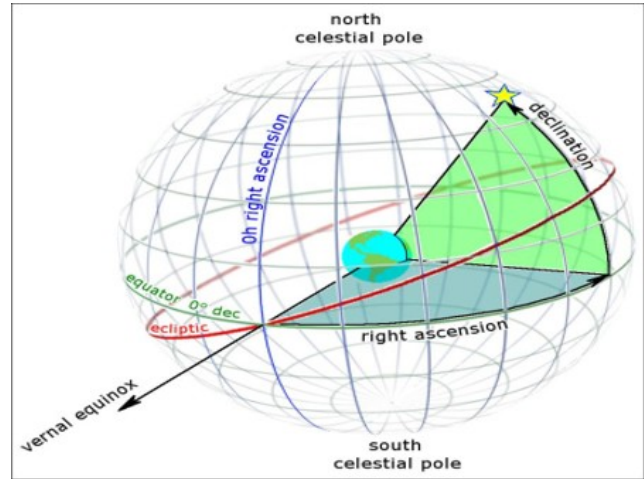


**Figure 1:** The Horizontal (Alt-Az) coordinate system showing altitude and azimuth for a star as seen by an observer.

### 2.2 2. Equatorial Coordinate System

The **equatorial system** is the most widely used coordinate system in astronomy. It is obtained by extending Earth's own latitude-longitude grid outward onto the celestial sphere, so the *celestial equator* is the projection of Earth's equator and the *celestial poles* are the projections of Earth's geographic poles. Unlike the horizontal system, equatorial coordinates change only very slowly over time (due to precession), making them suitable for star catalogues.

- **Right Ascension (RA)** — the celestial analogue of longitude. It is measured *eastwards* from the *vernal equinox* along the celestial equator. Unlike geographic longitude, RA is expressed in *hours, minutes, and seconds* ( $0^h$  to  $24^h$ ), because the sky appears to rotate through  $360^\circ$  in 24 hours.
- **Declination (Dec)** — the celestial analogue of latitude. It measures angular distance *north* (+) or *south* (−) of the celestial equator, expressed in degrees, arcminutes, and arcseconds ( $-90^\circ$  to  $+90^\circ$ ).

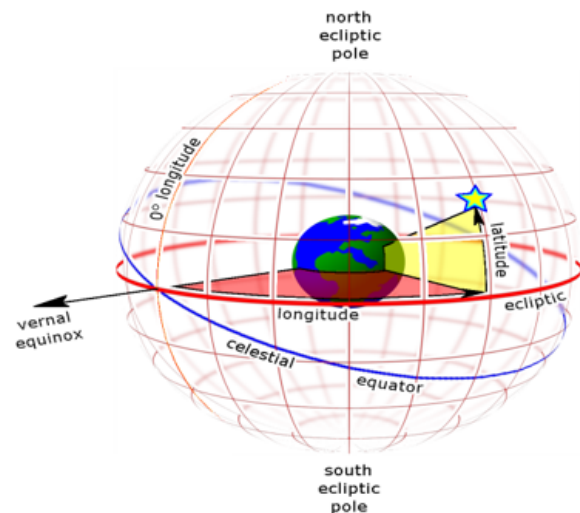


**Figure 2:** Equatorial coordinate system showing Right Ascension and Declination on the celestial sphere.

### 2.3 3. Ecliptic Coordinate System

This system uses the **ecliptic plane**—Earth's orbital plane around the Sun—as its reference plane. Because the ecliptic is inclined at  $23.5^\circ$  to the celestial equator, the ecliptic system is essentially the equatorial system tilted by  $23.5^\circ$ . It is particularly useful for describing Solar System objects.

- **Ecliptic Longitude** — measured eastward along the ecliptic from the vernal equinox; ranges  $0^\circ$  to  $360^\circ$ .
- **Ecliptic Latitude** — measured north (+) or south (−) of the ecliptic plane; ranges  $-90^\circ$  to  $+90^\circ$ .



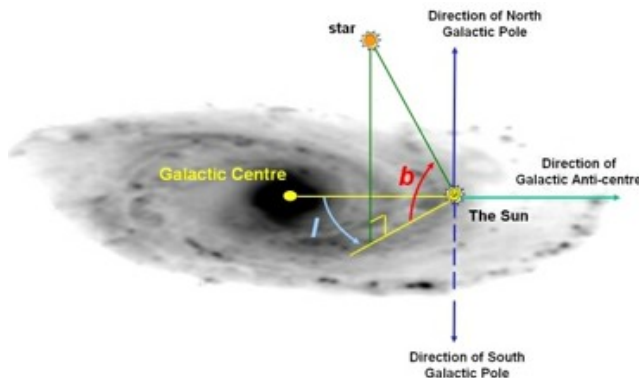
**Figure 3:** Ecliptic coordinate system, showing the ecliptic inclined  $23.5^\circ$  to the celestial equator.

### 2.4 4. Galactic Coordinate System

This system uses the **mid-plane of the Milky Way galaxy** as its reference plane. It is ideal for mapping objects within the Galaxy and studying large-scale galactic structure.

- **Galactic Longitude ( $\ell$ )** — measured in degrees; increases in the *eastward* direction from the direction of the galactic centre ( $\ell = 0^\circ$ ).

- **Galactic Latitude ( $b$ )** — measured north (+) or south (−) of the galactic plane; ranges  $-90^\circ$  (galactic south pole) to  $+90^\circ$  (galactic north pole).



**Figure 4:** Galactic coordinate system illustrating galactic longitude  $\ell$  and latitude  $b$  relative to the galactic centre and the Sun.

### 3. BASIC PHOTOMETRY

**Photometry** is the branch of observational astronomy that quantitatively measures the electromagnetic radiation received from celestial objects in order to derive useful physical information. By precisely measuring the brightness of stars, planets, galaxies, and other bodies, astronomers can determine fundamental physical properties including luminosity, temperature, distance, and size.

In practice, photometry is performed using **photometers** and **CCD (Charge-Coupled Device) cameras** attached to telescopes. Observations are made through standardised filter systems (such as Johnson–Cousins *UBVRI* or Sloan *ugriz*) that isolate specific wavelength bands, allowing precise and reproducible brightness comparisons across different objects and observing epochs.

#### Key applications of photometry:

- Measuring the brightness (apparent magnitude) of stars and other celestial objects.
- Determining the luminosity and surface temperature of stars via colour photometry.
- Studying variable stars and constructing their light curves (brightness vs. time).
- Detecting exoplanets using the *transit method* (measuring the small dip in stellar brightness as a planet crosses the stellar disk).
- Estimating distances to celestial objects using period–luminosity or flux–distance relationships.
- Comparing objects using standardised magnitude systems.



**Figure 5:** Multi-band photometric images of various celestial objects observed through different filters.

### 4. TYPES OF GALAXIES

Galaxies are enormous gravitationally bound systems composed of stars, stellar remnants, interstellar gas and dust, and dark matter. They come in a wide variety of shapes and structures that reflect their formation history, evolutionary processes, and interaction history with neighbouring galaxies.

**Spiral galaxies** are flat, rotating disk systems with a prominent central bulge and winding spiral arms. These arms are rich in young, hot blue stars, gas, and dust, making them active sites of ongoing star formation. The Milky Way is a classic example of a spiral galaxy.

**Barred spiral galaxies** are a variant of spirals in which a straight bar of stars runs through the central bulge. The spiral arms extend from the ends of this bar rather than directly from the nucleus. The bar structure is thought to channel gas efficiently toward the galactic centre, often fuelling bursts of star formation.

**Elliptical galaxies** have smooth, featureless, oval or spherical shapes with little to no internal structure. They contain mostly old, red, low-mass stars and very little cold gas or dust, so ongoing star formation is minimal. They range from nearly perfectly spherical (E0) to highly elongated forms (E7).

**Lenticular galaxies** (type S0) occupy an intermediate position between spirals and ellipticals. They possess a central bulge and a disk-like structure similar to spirals, but lack prominent spiral arms. They contain predominantly older stellar populations and have exhausted or lost most of their interstellar gas.

**Irregular galaxies** have no well-defined shape or regular structure. They often appear chaotic, and their gas-rich environment can support vigorous star formation. Many irregular galaxies owe their disrupted appearance to gravitational interactions or collisions with neighbouring systems.

**Peculiar galaxies** are those exhibiting unusual or distorted morphologies that do not fit neatly into standard classification schemes. Their irregular appearances are typically caused by tidal interactions, mergers, or collisions with other galaxies, which can dramatically distort stellar orbits and trigger intense star-formation episodes.

## 5. THE COSMIC WEB

**Cosmology** is the scientific study of the origin, large-scale structure, evolution, and ultimate fate of the universe. Modern cosmology traces the history of the universe from the *Big Bang* approximately 13.8 billion years ago, through inflation and the formation of the first atoms, to the emergence of stars, galaxies, and the intricate large-scale structure visible today. It also investigates the nature of **dark matter** and **dark energy**, which together constitute roughly 95% of the total energy content of the universe.

The **cosmic web** is the name given to the observed large-scale distribution of matter in the universe. Galaxies and galaxy clusters are not distributed randomly; instead they are arranged in an intricate network of *filaments* (elongated sheets and tendrils of matter) and *clusters* (dense nodes at filament intersections), separated by vast nearly empty regions called *voids*. This web-like pattern arose from the gravitational amplification of tiny quantum density fluctuations present in the very early universe.

Dark matter played the crucial role of forming the gravitational *scaffolding* that pulled ordinary (baryonic) matter into the filaments and clusters we observe today. Cosmological simulations such as the Millennium Simulation and IllustrisTNG reproduce the observed cosmic web with impressive fidelity, confirming our standard  $\Lambda$ CDM cosmological model.

## 6. STELLAR CYCLE

The **stellar cycle** describes the complete sequence of evolutionary stages a star passes through from its birth to its ultimate fate as a compact remnant. The precise pathway is determined primarily by the star's *initial mass*.

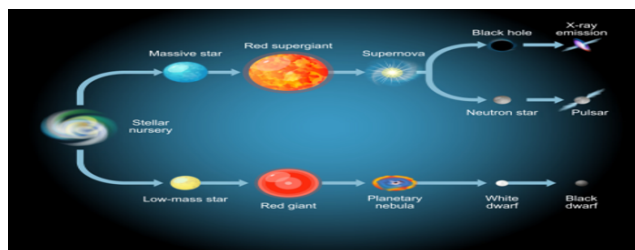
**Stage 1 — Stellar Nebula & Protostar.** All stars begin in a **stellar nebula**, a vast interstellar cloud composed predominantly of hydrogen and helium gas with traces of dust and heavier elements. Localised regions within the nebula can become gravitationally unstable and begin to collapse—perhaps triggered by a nearby supernova shock wave—forming a **protostar**: a dense, hot, opaque core that continues accreting mass from the surrounding envelope. As the protostar contracts, gravitational potential energy is converted to heat; when the core temperature reaches  $\sim 10^7$  K, hydrogen nuclei begin to undergo nuclear fusion.

**Stage 2 — Main Sequence.** When nuclear fusion of hydrogen into helium stabilises in the core, the star enters the **main sequence**—the longest and most stable phase of its life. The outward radiation pressure from fusion exactly balances inward gravitational collapse (hydrostatic equilibrium), keeping the star in a steady state. A star's luminosity, temperature, and lifespan on the main sequence are all determined by its mass.

**Stage 3 — Red Giant / Red Supergiant.** Once the hydrogen in the core is exhausted, the core contracts under gravity while the outer layers expand and cool dramatically, transforming the star into a **red giant** (for low/intermediate mass stars,  $M \lesssim 8 M_{\odot}$ ) or a **red supergiant** (for massive stars,  $M \gtrsim 8 M_{\odot}$ ). In this phase, helium and progressively heavier elements begin to fuse: helium fuses to carbon and oxygen via the triple-alpha process, and in massive stars this nucleosynthesis chain continues to produce elements up to iron.

**Low- and medium-mass stars ( $M \lesssim 8 M_{\odot}$ ).** The outer layers are expelled by stellar winds and pulsations, forming a glowing shell of ionised gas called a **planetary nebula**. The exposed core, composed mainly of carbon and oxygen, becomes a **white dwarf**: an Earth-sized, extremely dense object ( $\sim 10^6$  g/cm<sup>3</sup>) supported against further collapse by electron degeneracy pressure. The white dwarf slowly cools over billions of years, eventually becoming a theorised **black dwarf**.

**High-mass stars ( $M \gtrsim 8 M_{\odot}$ ).** Nuclear burning proceeds all the way to iron (Fe) in the core. Because iron cannot release energy by fusion, the core loses its thermal support and collapses catastrophically in a fraction of a second. The infalling outer layers rebound off the stiffened neutron-rich core, producing an enormous **supernova explosion** that briefly outshines an entire galaxy. The explosion disperses heavy elements into the interstellar medium. The remnant core collapses into either a **neutron star** (supported by neutron degeneracy pressure,  $\sim 10^{14}$  g/cm<sup>3</sup>) or, if the mass exceeds the Tolman–Oppenheimer–Volkoff limit, into a **black hole**, from which not even light can escape.



**Figure 6:** Stellar evolution diagram illustrating pathways for low-mass and high-mass stars from nebula to final remnant.

## 7. THE HR DIAGRAM

The **Hertzsprung–Russell (H–R) Diagram** is one of the most important tools in stellar astrophysics. It is a scatter plot of stars in the space of **luminosity** (y-axis, increasing upward) vs. **surface temperature** (x-axis, *decreasing* from left to right: hot stars on the left, cool stars on the right). Equivalently, one can plot absolute magnitude vs. spectral type or colour index.

**Key regions of the H–R diagram:**

- **Main Sequence** — a diagonal band running from the upper-left (hot, luminous O-type stars) to the



lower-right (cool, dim M-type red dwarfs). This is where hydrogen-burning stars spend the majority of their lives.

- **Giants and Supergiants** — located above and to the right of the main sequence. These are evolved stars with very large radii; despite their lower surface temperatures they can be extremely luminous.
- **White Dwarfs** — clustered in the lower-left corner. They are hot remnants of stellar cores but intrinsically dim because of their small size.

The H–R diagram is crucial because it encodes the *life cycle* of stars: a star’s position shifts across the diagram as it evolves, allowing astronomers to infer age, composition, and evolutionary stage from a single plot. Comparing observed clusters—in which all stars have the same age and distance—with theoretical isochrones on the H–R diagram is a primary method for determining stellar ages.

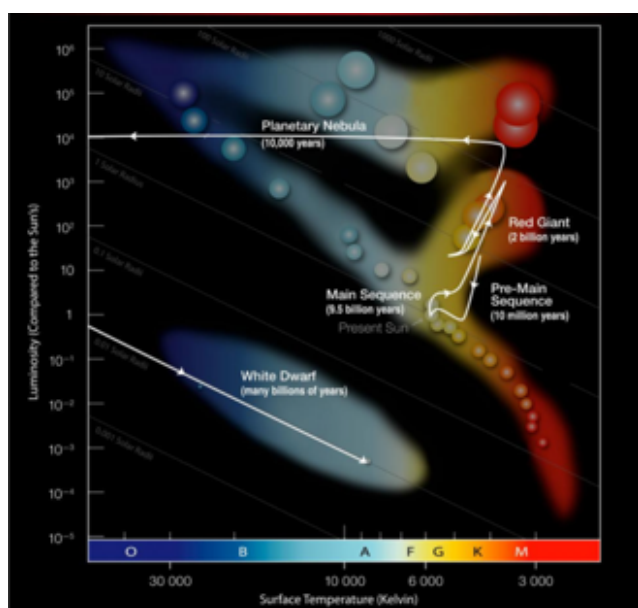


Figure 7: The Hertzsprung–Russell diagram showing the main sequence, giant branch, and white dwarf region.

## 8. EXOPLANET DETECTION & EXTRATERRESTRIAL LIFE

### 8.1 Extraterrestrial Life

Extraterrestrial life refers to any form of life—from simple microorganisms to complex intelligent beings—that might exist beyond Earth. The scientific motivation for this search rests on several pillars: the sheer vastness of the observable universe (containing  $\sim 10^{24}$  stars), the universality of chemistry, and the discovery that the basic ingredients for life as we know it (liquid water, carbon-based molecules, energy sources) are widespread throughout space.

Researchers focus particularly on **Mars** (evidence for ancient liquid water and subsurface ice), **Europa** (a global subsurface liquid water ocean beneath its ice shell, kept warm by tidal heating from Jupiter), and

**Enceladus** (active water-ice geysers detected by the Cassini spacecraft, suggesting a warm subsurface ocean in contact with a rocky seafloor). Although no confirmed evidence of extraterrestrial life has been found, the discovery of even microbial life elsewhere would be one of the most transformative scientific events in human history.

### 8.2 Exoplanet Detection Methods

Exoplanet detection is the process of identifying planets orbiting stars other than our Sun. Because exoplanets are tiny, cold, and situated next to blindingly bright host stars, direct detection is extremely challenging. Astronomers therefore rely on a suite of indirect and direct techniques.

#### 1. Transit Method

The most prolific technique. When a planet passes *in front of* its host star as seen from Earth, it blocks a small fraction of the starlight, causing a periodic, characteristic dip in the stellar brightness light curve. The depth of the dip gives the planet-to-star radius ratio, and the period between transits gives the orbital period. The Kepler mission used this method exclusively.

#### 2. Radial Velocity (Doppler) Method

A planet’s gravity causes its host star to execute a small reflex orbit around the system’s common centre of mass. This stellar wobble produces a periodic Doppler shift in the star’s spectral lines, alternately blue-shifted (star approaching) and red-shifted (star receding). The amplitude of the velocity variation gives a lower limit on the planet’s mass ( $M_p \sin i$ ), and the period gives the orbital period.

#### 3. Direct Imaging

In rare cases—typically for large, young, widely separated planets—powerful telescopes equipped with coronagraphs can block the stellar glare and directly photograph the planet. This yields spectra of the planet’s atmosphere but is currently limited to favourable geometries.

#### 4. Gravitational Microlensing

When a foreground star (with or without a planet) passes close to the line of sight toward a background star, the foreground star’s gravity temporarily brightens the background star. A planet around the lens star produces a characteristic short-duration anomaly in the magnification light curve. This method can detect planets at distances and masses inaccessible to other techniques.

### 8.3 Key Space Telescopes

| Telescope             | Key Contribution                                                                                                                                             |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kepler (2009–2018)    | Surveyed $\sim 150,000$ stars using transit photometry; confirmed $> 2,600$ exoplanets; showed small rocky planets are common.                               |
| TESS (2018–present)   | All-sky survey targeting bright nearby stars; continues discovering hundreds of new exoplanets suitable for follow-up.                                       |
| JWST (2021–present)   | Transmission and emission spectroscopy of exoplanet atmospheres; detects $\text{H}_2\text{O}$ , $\text{CO}_2$ , $\text{CH}_4$ , and potential biosignatures. |
| Hubble (1990–present) | Pioneering atmospheric studies; early evidence of water vapour and sodium in hot Jupiter atmospheres.                                                        |

**Table 1:** Major space telescopes used in exoplanet research.

Space telescopes operate above Earth’s atmosphere, avoiding the atmospheric turbulence and absorption that limit ground-based observatories. This allows detection of the sub-millimagnitude brightness variations produced by transiting Earth-sized planets. Today, astronomers have confirmed more than **5,000** exoplanets.

## 9. LUNAR SCIENCE

### 9.1 Moon’s History and Origin

The Moon is Earth’s only natural satellite, believed to have formed approximately **4.5 billion years ago**, around the same time as Earth itself. Its bulk composition and isotopic ratios are strikingly similar to Earth’s, providing a key constraint on formation models.

#### Similarities between Earth and the Moon:

- Both are rocky bodies composed mainly of silicate rocks and metals.
- Both have geological features: mountains, valleys, plains, and impact craters.
- Moon rocks returned by Apollo missions have oxygen isotope ratios identical to terrestrial rocks—a uniquely close match not shared by other Solar System bodies.
- Both formed in the inner Solar System roughly 4.5 Gya.

The **Giant Impact Hypothesis** is the most widely accepted model for lunar origin. About 4.5 billion years ago, a Mars-sized protoplanet called **Theia** collided with the young Earth at a glancing angle. The collision ejected vast amounts of molten silicate material into Earth orbit; this debris rapidly coalesced under self-gravity to form the Moon. The model naturally explains the Moon’s small iron core, its depletion in volatile elements, and its identical oxygen isotope signature.

#### Brief geological history of the Moon:

- **Formation ( $\sim 4.5$  Gya)** — Moon accretes from Theia-impact debris.
- **Magma Ocean Stage ( $\sim 4.5$ – $4.3$  Gya)** — the entire surface is covered by a global magma ocean; the crust solidifies as plagioclase feldspar floats to

the top.

- **Late Heavy Bombardment ( $\sim 4.1$ – $3.8$  Gya)** — a spike in the impact flux creates large multi-ring basins (e.g. Imbrium, Orientale).
- **Volcanic Activity ( $\sim 3.9$ – $1.0$  Gya)** — vast outpourings of basaltic lava flood the impact basins, creating the dark smooth plains known as *lunar maria*.
- **Present Day** — the Moon is essentially geologically inactive. It continues to orbit Earth, raising ocean tides and stabilising Earth’s obliquity, which helps maintain long-term climate stability.

### 9.2 Physical Properties of the Moon

| Property            | Value / Description                                            |
|---------------------|----------------------------------------------------------------|
| Classification      | Earth’s only natural satellite; 5th largest in Solar System    |
| Mean Earth distance | 384 400 km (semi-major axis)                                   |
| Diameter            | 3 474 km ( $\approx \frac{1}{4}$ of Earth’s diameter)          |
| Mass                | $7.34 \times 10^{22}$ kg ( $\approx \frac{1}{81}$ of Earth)    |
| Surface gravity     | 1.62 m/s <sup>2</sup> ( $\approx \frac{1}{6}$ of Earth’s)      |
| Orbital period      | 27.3 days (sidereal); 29.5 days (synodic)                      |
| Surface temperature | +127°C (lunar noon) to $-173^\circ\text{C}$ (lunar midnight)   |
| Atmosphere          | Negligible exosphere; no weather, wind, or rain                |
| Light source        | Reflected sunlight (albedo $\approx 0.12$ )                    |
| Rotation            | Tidally locked: same hemisphere always faces Earth             |
| Water ice           | Confirmed in permanently shadowed polar craters (LCROSS, 2009) |

**Table 2:** Key physical properties of the Moon.

### 9.3 Other Notable Moons of the Solar System

Beyond Earth’s Moon, the Solar System hosts an extraordinary diversity of natural satellites, several of which are of tremendous astrobiological and geological interest.

**Ganymede (Jupiter)** is the largest moon in the Solar System, exceeding even Mercury in diameter. It is the only moon known to have its own internally generated magnetic field.

**Europa (Jupiter)** is covered by a smooth, geologically young ice shell beneath which lies a global salt-water ocean kept liquid by tidal flexing from Jupiter. It is considered one of the most promising locations in the Solar System to search for extant microbial life, and will be investigated in detail by NASA’s Europa Clipper mission.

**Titan (Saturn)** is the second-largest moon in the Solar System and the only moon with a dense, planet-like nitrogen-rich atmosphere. It hosts lakes, rivers, and rain of liquid methane and ethane, making it a striking analogue of an early Earth—but at  $-179^\circ\text{C}$ .

**Enceladus (Saturn)**, despite its small size, is one of the most geologically active bodies in the Solar System. Cassini discovered active plumes of water vapour, ice particles, organic molecules, and molecular hydrogen erupting from its south polar region, indicating a

warm subsurface ocean in contact with a rocky seafloor—conditions potentially suitable for life.

**Io (Jupiter)** is the most volcanically active body in the Solar System, with hundreds of active volcanoes driven by intense tidal heating from Jupiter. Its surface is continuously resurfaced by sulphurous lava flows.

**Callisto (Jupiter)** is one of the most heavily cratered objects in the Solar System, suggesting a very ancient surface. Magnetic induction measurements hint at a possible subsurface saline ocean.

**Triton (Neptune)** is unique in being the only large moon in the Solar System with a *retrograde* orbit (opposite to Neptune’s rotation), suggesting it was captured from the Kuiper Belt. It has active nitrogen geysers and is slowly spiralling inward, destined to be disrupted by tidal forces in  $\sim 3.6$  billion years.

## 10. MACHINE LEARNING

**Machine Learning (ML)** is a subfield of Artificial Intelligence (AI) concerned with developing algorithms that can *learn* statistical patterns from data and use those patterns to make decisions or predictions on new, unseen data—without being explicitly programmed for every possible scenario.

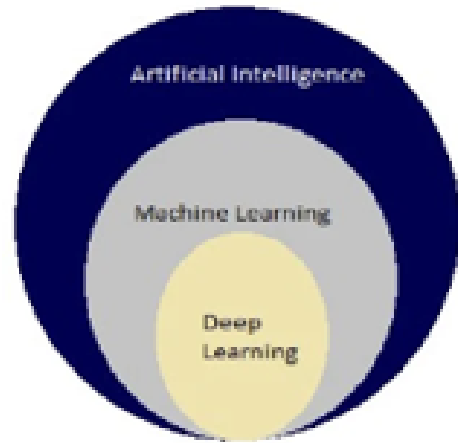
The central objective of supervised ML is to estimate an unknown target function:

$$y = f(x)$$

where  $x \in \mathbb{R}^n$  is the input feature vector,  $y$  is the target output, and  $f$  is the mapping learned from a labelled training dataset.

**Relationship between AI, ML, and Deep Learning:**

- **AI** — the broad field of building systems that exhibit intelligent behaviour.
- **ML** — a subset of AI in which systems *learn from data* rather than following hand-crafted rules.
- **Deep Learning** — a subset of ML that uses *multi-layer neural networks* to automatically learn hierarchical feature representations.



**Figure 8:** Nested relationship:  $\text{AI} \supset \text{Machine Learning} \supset \text{Deep Learning}$ .

### 10.1 A. Supervised Learning

In supervised learning, the model is trained on a **labelled dataset**  $\{(x_i, y_i)\}$  where the correct output  $y_i$  is known for each input  $x_i$ . The model iteratively adjusts its parameters to minimise the discrepancy (loss) between its predictions and the correct labels. There are two principal subtypes:

- **Classification** — the output is a discrete class label. Examples: spam detection (spam/not spam); image recognition (cat/dog); lunar feature classification (crater/non-crater).
- **Regression** — the output is a continuous numerical value. Examples: predicting house prices; estimating stellar ages from photometry; inferring crater depth from shadow length.

### 10.2 B. Unsupervised Learning

In unsupervised learning, only the input data  $X$  is available—no labels are provided. The algorithm must discover **hidden structure** within the data autonomously.

- **Clustering** — partition data points into groups such that intra-cluster similarity is maximised and inter-cluster similarity is minimised. Example: grouping lunar surface patches by texture to distinguish crater floors, ejecta blankets, and smooth maria.
- **Dimensionality Reduction** — project high-dimensional data into a lower-dimensional space while preserving maximal information. Example: Principal Component Analysis (PCA) finds the orthogonal directions of maximum variance, enabling visualisation and compression of hyperspectral imagery.

### 10.3 C. Reinforcement Learning (RL)

In reinforcement learning, an **agent** learns to make sequential decisions by interacting with an **environment** and receiving scalar **reward** signals. The agent’s goal is to learn a *policy* (a mapping from states to actions) that maximises expected cumulative long-term reward.

| Component        | Description                                                            |
|------------------|------------------------------------------------------------------------|
| Agent            | The learner / decision maker (e.g. robot, autonomous rover).           |
| Environment      | Everything outside the agent; the world it acts upon.                  |
| State ( $s$ )    | Representation of the current situation (position, velocity, sensors). |
| Action ( $a$ )   | A decision the agent can take at each time step.                       |
| Reward ( $r$ )   | Scalar signal indicating how good the action was.                      |
| Policy ( $\pi$ ) | The agent's strategy: mapping from states to actions.                  |

**Table 3:** Components of a Reinforcement Learning system.

## 11. THE MACHINE LEARNING PIPELINE

Building a production-quality ML model is a systematic, multi-stage process that goes far beyond simply calling `model.fit()`. The standard pipeline consists of the following steps:

1. **Task Recognition** — clearly define the problem: what is the input, what is the output, and what metric defines success?
2. **Dataset Collection** — ML is fundamentally data-driven. High-quality, representative data is often more important than algorithmic sophistication. Sources include databases, web scraping, sensors, and open-source repositories (Kaggle, NASA PDS).
3. **Data Cleaning & Preprocessing** — handle missing values, remove outliers, encode categorical variables, normalise and standardise numerical features.
4. **Exploratory Data Analysis (EDA)** — visualise and summarise the data before modelling using histograms, scatter plots, box plots, and correlation heatmaps to understand distributions, outliers, and feature relationships.
5. **Feature Engineering** — create new, informative inputs from raw data. Examples: combining raw features (Area = Length  $\times$  Width); extracting temporal components (day of week, is-weekend); computing polynomial or interaction terms; extracting image statistics.
6. **Model Selection & Loss Definition** — choose the appropriate algorithm based on task type and dataset size. Simple problems: Linear Regression, Logistic Regression, Decision Trees. Complex tabular data: Random Forest, XGBoost, LightGBM. Image / text / audio: Deep Learning (CNNs, RNNs, Transformers). Define a differentiable *loss function* appropriate to the task (e.g. mean squared error for regression, categorical cross-entropy for multi-class classification).
7. **Train / Validation / Test Split** — the dataset must be partitioned to enable unbiased evaluation. The **training set** is used to update model parameters (weights). The **validation set** is used during training to tune hyperparameters and monitor for overfitting. The **test set** is held out completely and used *once* at the very end to report the final generalisation performance.
8. **Model Training** — pass data through the model, compute the loss, compute gradients via backpropagation, and update parameters using an optimiser (e.g. Adam, SGD). This loop repeats for multiple *epochs*.
9. **Evaluation (Final Metrics)** — assess model performance on the held-out test set using appropriate metrics (accuracy, F1-score, AUC-ROC, confusion matrix). Diagnose whether the model is underfitting (high bias) or overfitting (high variance).
10. **Inference & Deployment** — once validated, the trained model (with frozen weights) is deployed. It must apply the *identical* preprocessing pipeline used during training to incoming real-time data, and output predictions with low latency.

## 12. CONVOLUTIONAL NEURAL NETWORKS (CNN)

A CNN image classification model was developed to classify images into six terrain categories: **buildings, forest, glacier, mountain, sea, and street**. This section analyses the core architectural concepts and summarises the end-to-end workflow.

### 12.1 Part 1: CNN Architecture Concepts

#### Core Layer Types

The model uses a **Sequential** stack of layers designed to handle high-dimensional spatial data.

- **Convolutional Layers (Conv2D)** — apply learnable  $3 \times 3$  filter kernels that slide across the input image, computing dot products to produce *feature maps*. Early layers detect low-level features (edges, corners, colour gradients); deeper layers detect progressively more abstract, high-level semantic patterns. The model uses three convolutional blocks with 32, 64, and 128 filters respectively.
- **Activation Function (ReLU)** — Rectified Linear Unit:  $f(x) = \max(0, x)$ . Applied element-wise after convolution, ReLU introduces non-linearity, enabling the network to learn complex decision boundaries rather than simple linear mappings.
- **Batch Normalisation (BatchNorm)** — normalises the output of each convolutional layer to have zero mean and unit variance across the current mini-batch. Placed immediately after each convolution, it stabilises training, reduces *internal covariate shift*, allows use of higher learning rates, and acts as a mild regulariser.
- **Pooling Layers (MaxPooling2D)** — perform progressive spatial downsampling using  $2 \times 2$  windows, retaining only the maximum activation in each window. This halves the spatial dimensions at each stage ( $64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$ ), reducing computational cost and helping the model achieve *translational invariance*.
- **Flatten Layer** — converts the final  $8 \times 8 \times 128$  3-D feature tensor into a 1-D vector of 8,192 values,



serving as input to the fully connected classifier head.

- **Dense / Fully Connected Layers** — a hidden layer with 256 neurons (ReLU) captures global feature correlations. The final output layer has 6 neurons with **softmax** activation, producing a probability distribution over the 6 classes.

#### Regularisation (Fighting Overfitting)

- **Dropout (Dropout(0.4))** — during each forward pass in training, randomly sets 40% of the neurons in the dense layer to zero. This prevents co-adaptation between neurons, forcing the network to learn more robust and generalisable feature representations.
- **Data Augmentation (ImageDataGenerator)** — artificially expands the effective training set by applying stochastic transformations (rotations up to 15°, horizontal/vertical shifts of 10%, zooms of 10%, horizontal flips) to training images on-the-fly, dramatically reducing overfitting.

#### Model block structure:

1. Block 1: Conv2D(32) → BatchNorm → MaxPool
2. Block 2: Conv2D(64) → BatchNorm → MaxPool
3. Block 3: Conv2D(128) → BatchNorm → MaxPool
4. Classifier: Flatten → Dense(256, ReLU) → Dropout(0.4) → Dense(6, Softmax)

Total parameters: **2,193,094** (2,192,646 trainable).

### 12.2 Part 2: Workflow Summary

#### Step 1 — Environment Setup

Essential packages are loaded: **numpy** and **pandas** (array operations), **matplotlib** and **cv2** (visualisation and image I/O), **sklearn** (preprocessing utilities), and **tensorflow.keras** (model building and training).

#### Step 2 — Directory Profiling & Class Mapping

Paths to the three dataset splits (**seg\_train**, **seg\_test**, **seg\_pred**) are mapped, and folder structure is parsed to identify the six target classes: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street'].

#### Step 3 — Model Assembly

The CNN architecture described above is declared as a **Sequential** container. Feature map dimensions evolve through the network as follows:  $(64 \times 64 \times 32) \rightarrow (32 \times 32 \times 64) \rightarrow (8 \times 8 \times 128) \rightarrow 8,192$ -dim vector.

#### Step 4 — Optimisation & Training

The network is compiled with the **Adam optimiser** (adaptive learning rates per parameter) and **categorical cross-entropy loss**. Training results over 20 epochs:

| Epoch | Train Accuracy | Val Accuracy               |
|-------|----------------|----------------------------|
| 1     | ~58.35%        | Val loss = 6.36 (unstable) |
| 5     | —              | ~77.67% / loss = 0.64      |
| 20    | 85.45%         | 75.03%                     |

**Table 4:** Terrain CNN training trajectory over 20 epochs.

## 13. TASK: CNN EMOTION RECOGNITION

**Objective:** Apply the CNN framework to classify human facial expressions from greyscale photographs into 7 classes: **angry**, **disgust**, **fear**, **happy**, **neutral**, **sad**, **surprise**.

**Dataset:** Kaggle Face Expression Recognition Dataset ([jonathanoheix/face-expression-recognition-dataset](https://kaggle.com/jonathanoheix/face-expression-recognition-dataset)) — 28,821 training images and 7,066 test images, all  $64 \times 64$  px greyscale.

### 13.1 Annotated Code Walkthrough

#### Step 1 — Imports

```
import numpy as np
import matplotlib.pyplot as plt
import os, cv2
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (Conv2D,
    MaxPooling2D, Flatten, Dense,
    Dropout, BatchNormalization)
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image \
    import ImageDataGenerator
```

**Notes:** **numpy** — matrix and mathematical operations; **matplotlib.pyplot** — visualisation and plots; **cv2** — computer vision library for image reading and resizing; **sklearn.preprocessing** — **LabelEncoder** for converting string class names to integers; **tensorflow.keras** — Google's open-source deep learning platform (Keras is the high-level API providing **.models**, **.layers**, **.utils**, and **.preprocessing** modules for model building, training, and data augmentation).

#### Step 2 — Dataset Loading & Preprocessing

```
BASE_DIR = '/kaggle/input/.../face-expression-...'
TRAIN_DIR = os.path.join(BASE_DIR, 'images', 'train')
TEST_DIR = os.path.join(BASE_DIR, 'images',
    'validation')

CLASSES = sorted(os.listdir(TRAIN_DIR))
# Output: ['angry', 'disgust', 'fear', 'happy',
#         'neutral', 'sad', 'surprise']

IMG_SIZE = 64

def load_data(directory):
    images, labels = [], []
    for cls in CLASSES:
        folder = os.path.join(directory, cls)
        for fname in os.listdir(folder):
            img = cv2.imread(
                os.path.join(folder, fname),
                cv2.IMREAD_GRAYSCALE)
            if img is None: continue
            img = cv2.resize(img,
                (IMG_SIZE, IMG_SIZE))
            images.append(img)
            labels.append(cls)
    return np.array(images), np.array(labels)

X_tr, y_tr = load_data(TRAIN_DIR)
X_te, y_te = load_data(TEST_DIR)

X_train = np.expand_dims(X_tr, axis=-1) / 255.0
X_test = np.expand_dims(X_te, axis=-1) / 255.0

le = LabelEncoder()
y_train = to_categorical(le.fit_transform(y_tr))
y_test = to_categorical(le.transform(y_te))

# X_train: (28821, 64, 64, 1) | y_train: (28821, 7)
# X_test: (7066, 64, 64, 1) | y_test: (7066, 7)
```

**Notes:** Each image is read as greyscale, resized to  $64 \times 64$

px, then normalised by dividing by 255 to bring pixel values to  $[0, 1]$ . `np.expand_dims` adds a channel dimension:  $(64, 64) \rightarrow (64, 64, 1)$ . `LabelEncoder` maps class strings to integers; `to_categorical` converts integers to one-hot vectors of length 7. There are 28,821 training images and 7,066 test images in total.



Figure 9: One sample image from each of the 7 emotion classes in the training set.

### Step 3 — Model Architecture

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu',
        padding='same',
        input_shape=(IMG_SIZE, IMG_SIZE, 1)),
    BatchNormalization(),
    MaxPooling2D(2, 2),

    Conv2D(64, (3,3), activation='relu',
        padding='same'),
    BatchNormalization(),
    MaxPooling2D(2, 2),

    Conv2D(128, (3,3), activation='relu',
        padding='same'),
    BatchNormalization(),
    MaxPooling2D(2, 2),

    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.4),
    Dense(len(CLASSES), activation='softmax')
])
```

Notes: This is the same three-block architecture discussed in Section 9. The final `Dense` layer now has `len(CLASSES) = 7` output neurons, one per emotion class. The `softmax` activation ensures the outputs sum to 1, forming a valid probability distribution over classes.

### Step 4 — Data Augmentation

```
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    zoom_range=0.1)

sample_batch = X_train[:9]
aug_iter = datagen.flow(sample_batch,
                        batch_size=9,
                        shuffle=False)

aug_batch = next(aug_iter)

fig, axes = plt.subplots(2, 9, figsize=(18,4))
for i in range(9):
    axes[0,i].imshow(sample_batch[i], cmap='gray')
    axes[0,i].axis('off')
    axes[1,i].imshow(
        np.clip(aug_batch[i], 0, 1), cmap='gray')
    axes[1,i].axis('off')
axes[0,0].set_ylabel("Original", fontsize=10)
axes[1,0].set_ylabel("Augmented", fontsize=10)
plt.tight_layout(); plt.show()
```

Notes: The `ImageDataGenerator` applies random rotations (up to  $15^\circ$ ), width and height shifts (up to 10% of image size), horizontal flips, and zooms (up to 10%) on-the-fly during training batches. These transformations artificially expand the training set diversity, preventing overfitting by ensuring the model rarely sees the exact same pixel layout twice.

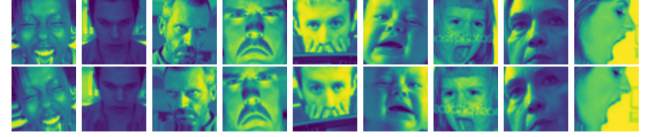


Figure 10: Top row: original training images. Bottom row: corresponding augmented versions showing random rotations, shifts, flips, and zooms.

### Step 5 — Compilation & Training

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'])

history = model.fit(
    datagen.flow(X_train, y_train, batch_size=64),
    epochs=50,
    validation_data=(X_test, y_test))

# Epoch 34 results:
# Train acc: 0.6783 | Val acc: 0.6684
# Train loss: 0.8642 | Val loss: 0.9061
```

Notes: `datagen.flow` feeds augmented mini-batches to the model during training. The **Adam optimiser** (Adaptive Moment Estimation) maintains per-parameter adaptive learning rates using first and second moment estimates of the gradients, making it robust and fast-converging for image classification tasks. **Categorical cross-entropy** is the standard loss function for multi-class classification with one-hot encoded labels:  $\mathcal{L} = -\sum_k y_k \log \hat{p}_k$ .

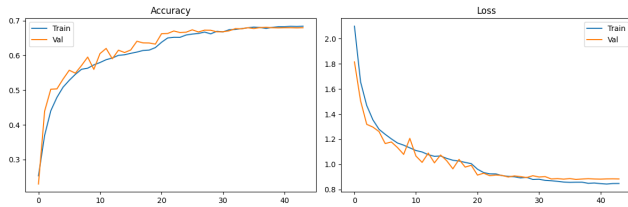


Figure 11: Training log output showing accuracy and loss values at epoch 34 of training.

### Step 6 — Accuracy & Loss Curves

```
fig, axes = plt.subplots(1, 2, figsize=(12,4))
axes[0].plot(history.history['accuracy'],
              label='Train')
axes[0].plot(history.history['val_accuracy'],
              label='Val')
axes[0].set_title('Accuracy')
axes[0].legend()
axes[1].plot(history.history['loss'],
              label='Train')
axes[1].plot(history.history['val_loss'],
              label='Val')
axes[1].set_title('Loss')
axes[1].legend()
plt.tight_layout(); plt.show()
```

Notes: Ideally, training accuracy should increase and training loss should decrease monotonically with epochs. The gap between training and validation curves indicates the degree of overfitting; data augmentation and dropout help keep this gap small.



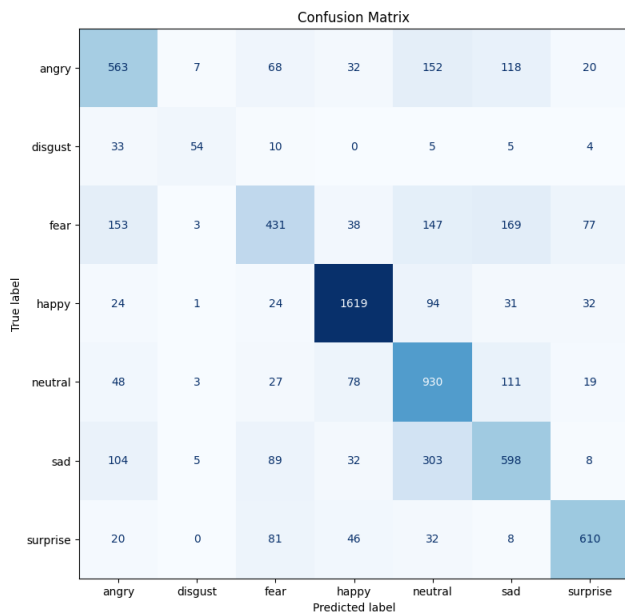
**Figure 12:** Training and validation accuracy (left) and loss (right) over 50 epochs.

### Step 7 — Confusion Matrix

```
from sklearn.metrics import (confusion_matrix,
                             ConfusionMatrixDisplay)

y_pred = model.predict(X_test)
cm = confusion_matrix(
    y_te,
    le.inverse_transform(
        np.argmax(y_pred, axis=1)))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=le.classes_)
fig, ax = plt.subplots(figsize=(8,8))
disp.plot(ax=ax, colorbar=False, cmap='Blues')
plt.title("Confusion Matrix")
plt.tight_layout(); plt.show()
```

*Notes:* The confusion matrix visualises the actual vs. predicted class labels across all 7 emotion categories. Diagonal entries represent correct predictions; off-diagonal entries reveal which classes the model confuses with one another, providing diagnostic information for model improvement.

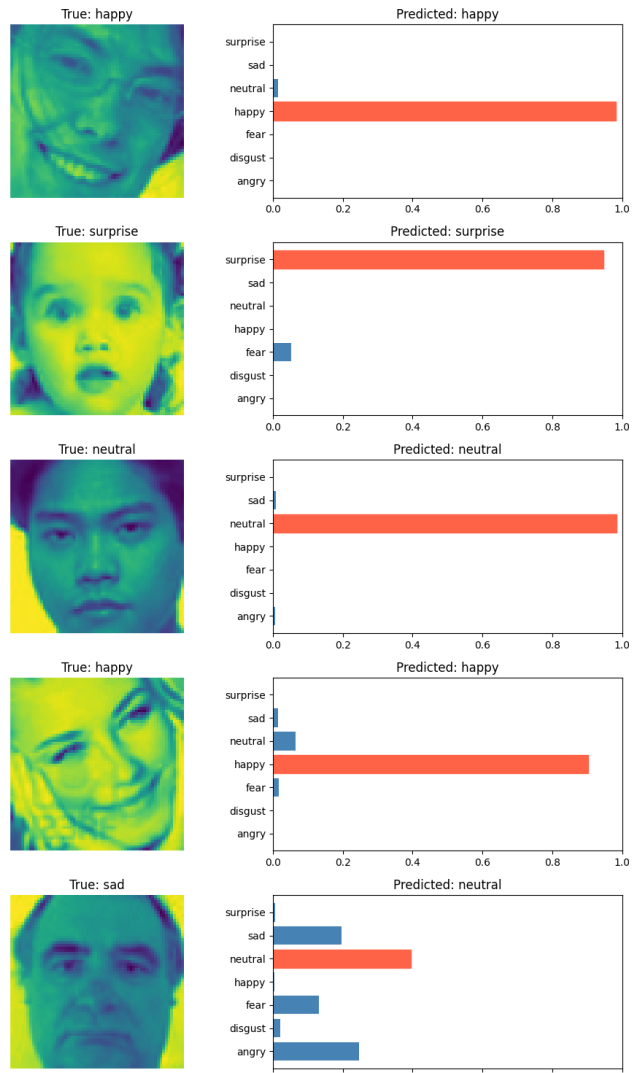


**Figure 13:** Confusion matrix for the 7-class emotion recognition model on the held-out test set.

### Step 8 — Sample Predictions

```
n = 10
indices = np.random.randint(len(X_test), size=n)
fig, axes = plt.subplots(n, 2,
                        figsize=(10, n*3))
for row, idx in enumerate(indices):
    img = X_test[idx]
    probs = model.predict(
        img[np.newaxis,...], verbose=0)[0]
    axes[row,0].imshow(img, cmap='gray')
    axes[row,0].set_title(
        f"True: {y_te[idx]}")
    axes[row,0].axis('off')
```

```
bars = axes[row,1].barh(
    le.classes_, probs, color='steelblue')
bars[np.argmax(probs)].set_color('tomato')
axes[row,1].set_xlim(0, 1)
axes[row,1].set_title(
    f"Pred: {le.classes_[np.argmax(probs)]}")
plt.tight_layout(); plt.show()
```



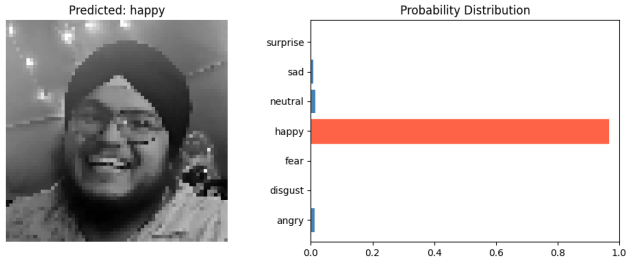
**Figure 14:** Randomly selected test images with true labels and predicted class-probability bar charts. The predicted class bar is highlighted in red.

### Step 9 — Custom Image Inference

```
custom_img_path = '../Harpuneet.jpeg'
raw_img = cv2.imread(custom_img_path,
                    cv2.IMREAD_GRAYSCALE)
if raw_img is None:
    print("Error: image not found.")
else:
    resized = cv2.resize(raw_img, (64,64))
    normalised = resized / 255.0
    tensor = normalised[np.newaxis,...,
                        np.newaxis]
    probs = model.predict(tensor,
                        verbose=0)[0]
    fig, axes = plt.subplots(1, 2,
                        figsize=(10,4))
    axes[0].imshow(normalised, cmap='gray')
    axes[0].set_title(
        f"Pred: {le.classes_[np.argmax(probs)]}")
    axes[0].axis('off')
    bars = axes[1].barh(le.classes_, probs,
                        color='steelblue')
    bars[np.argmax(probs)].set_color('tomato')
    axes[1].set_xlim(0, 1)
```

```
axes[1].set_title("Probability Distribution")
plt.tight_layout(); plt.show()
# Result: Predicted 'happy' correctly.
```

*Notes:* The trained model was applied to a real-life photograph not seen during training. It successfully predicted the correct facial expression (*happy*), demonstrating reasonable generalisation beyond the training distribution.



**Figure 15:** Real-life custom image with predicted class label and per-class probability distribution bar chart. Model correctly predicts *happy*.

## 14. DATASETS USED

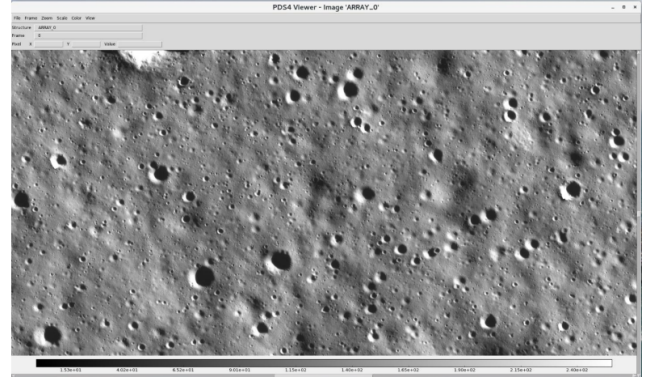
The primary data source for this project is the **PRADAN** (Planetary Research and Analysis Data Access Node) portal, a web application developed by the **Indian Space Science Data Centre (ISSDC)** to provide scientists and the public with seamless access to data archived from ISRO's space missions. Portal: [pradan.issdc.gov.in/ch2/](http://pradan.issdc.gov.in/ch2/).

Two Chandrayaan-II instrument datasets are used in this project:

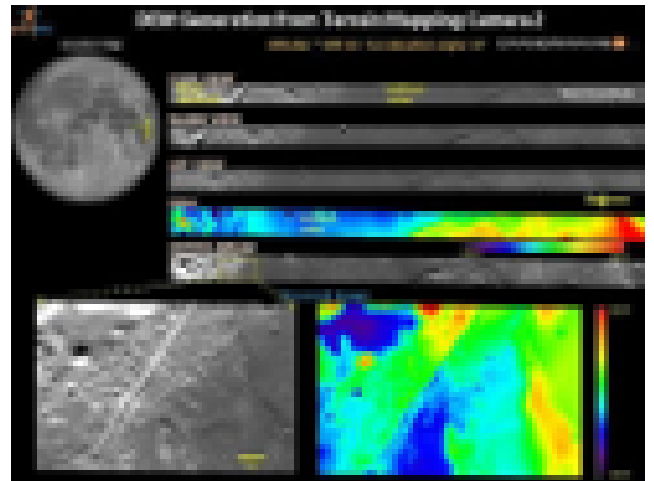
- **OHRC (Optical High Resolution Camera)** — provides very high spatial resolution panchromatic images of the lunar surface, used to identify crater morphology and assess moonslide risk. Detailed user-guide PDFs giving file hierarchy, data formats, and required software are available from the PRADAN portal.
- **TMC-2 (Terrain Mapping Camera 2)** — provides stereo images enabling the construction of Digital Elevation Models (DEMs) in *.tif* format, used to quantify terrain slopes and elevation profiles near identified craters. The rainbow-coloured DEM images represent different height values.

|                    | OHRC                                              | TMC-2                                              |
|--------------------|---------------------------------------------------|----------------------------------------------------|
| <b>Full name</b>   | Optical High Resolution Camera                    | Terrain Mapping Camera 2                           |
| <b>Data type</b>   | High-resolution panchromatic lunar surface images | Stereo images + DEM ( <i>.tif</i> ) elevation maps |
| <b>Project use</b> | Identify craters; assess moonslide risk           | Quantify terrain slope near craters                |
| <b>Formats</b>     | <i>.png .xml .csv .img .lbr .oat .oath .spr</i>   | <i>.tif .png .xml .csv .img .lbr .oat .spr</i>     |
| <b>Guide</b>       | <a href="#">OHRC User Guide</a>                   | <a href="#">TMC-2 User Guide</a>                   |

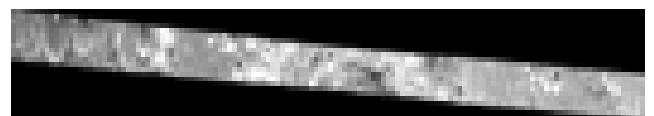
**Table 5:** Comparison of the OHRC and TMC-2 Chandrayaan-II datasets from PRADAN.



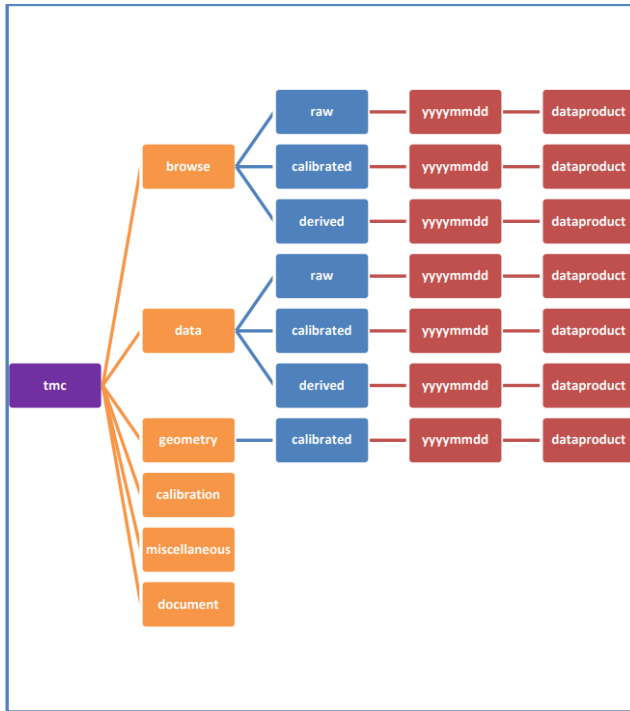
**Figure 16:** Sample OHRC high-resolution panchromatic image of the lunar surface showing crater morphology used to assess moonslide risk.



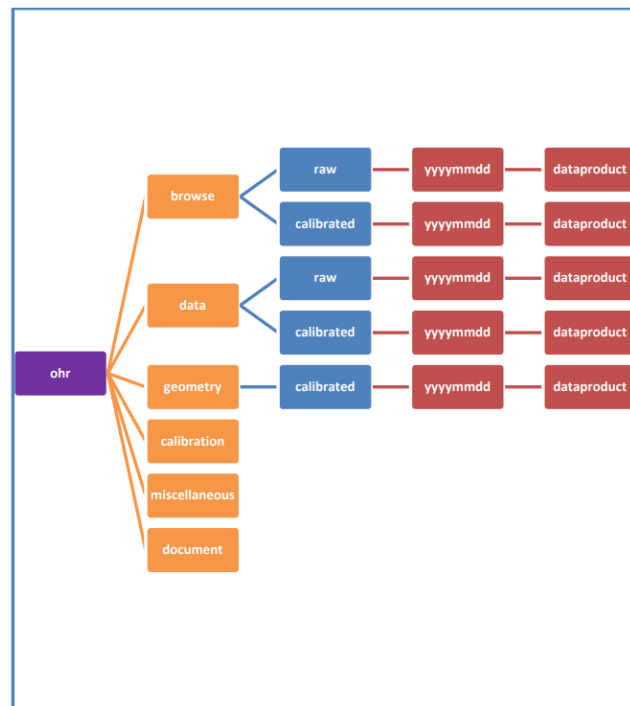
**Figure 17:** Sample TMC-2 Digital Elevation Model (DEM) image. Rainbow colouring represents terrain height; blue = low elevation, red = high elevation.



**Figure 18:** TMC-2 strip image showing a swath of the lunar surface as acquired in pushbroom imaging mode.



**Figure 19:** OHRC dataset file hierarchy as shown in the PRADAN portal, illustrating the directory structure of a downloaded data product.



**Figure 20:** TMC-2 dataset file hierarchy as shown in the PRADAN portal, illustrating the directory structure of a downloaded data product.

## 15. ASSIGNMENT LINKS & MENTEE SUBMISSIONS

| Resource                         | Link                                                                                                                                                |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Reference Code                   | <a href="https://tanmayshukla05/image-classification-using-cnn-task">.../tanmayshukla05/image-classification-using-cnn-task</a>                     |
| Kaggle Dataset                   | <a href="https://jonathanoheix/face-expression-recognition-dataset">.../jonathanoheix/face-expression-recognition-dataset</a>                       |
| <b>Mentee Kaggle Submissions</b> |                                                                                                                                                     |
| Niharika                         | <a href="https://pathriniharika/fer-cnn1">.../pathriniharika/fer-cnn1</a>                                                                           |
| Sayantan                         | <a href="https://sayantanmondal1317/moonslide-task1-v-1-1">.../sayantanmondal1317/moonslide-task1-v-1-1</a><br>(main code for the task given above) |
| Arnav                            | <a href="https://arnavps/notebook54b884a0d2">.../arnavps/notebook54b884a0d2</a>                                                                     |
| Mayank                           | <a href="https://mayankq2007/face-expression-recognition-cnn">.../mayankq2007/face-expression-recognition-cnn</a>                                   |

**Table 6:** Reference material and mentee submission links.

## 16. MID-EVALUATION PROGRESS SUMMARY

The following topics and tasks have been completed in the first half of the project:

- ✓ **Background astronomy:** Celestial sphere and all four coordinate systems (horizontal, equatorial, ecliptic, galactic).
- ✓ **Observational astronomy:** Basic photometry, magnitude systems, photometric instrumentation.
- ✓ **Extragalactic astronomy:** Types of galaxies; the cosmic web; cosmology overview.
- ✓ **Stellar physics:** Complete stellar cycle; H–R diagram.
- ✓ **Planetary science:** Exoplanet detection methods; key telescopes; extraterrestrial life motivation.
- ✓ **Lunar science:** Moon’s origin (Giant Impact Hypothesis), geological history, physical properties, notable Solar System moons.
- ✓ **Machine Learning:** Supervised, unsupervised, and reinforcement learning; the complete 10-step ML pipeline.
- ✓ **CNN and computer vision:** Architecture layers (Conv2D, BatchNorm, MaxPool, Dense, Dropout, Softmax); regularisation; data augmentation.
- ✓ **Hands-on task:** CNN implemented for facial emotion classification (7 classes, 28K+ images); confusion matrix; custom image inference.
- ✓ **Chandrayaan-II datasets:** OHRC and TMC-2 datasets accessed and explored via the PRADAN portal.

### Planned work for the second half of the project:

- Apply CNN-based detection directly to OHRC lunar imagery to identify crater rims and landslide-prone regions.
- Use TMC-2 DEM data to extract slope, aspect, and roughness metrics for candidate landslide sites.
- Integrate GIS tools (QGIS) for geo-referenced spatial mapping of detected features.
- Create a preliminary catalogue of landslide-prone lunar regions.
- Explore transfer learning to boost lunar detection performance.